

Church App

Documentation backend (Supabase)

Backend — Supabase

Doc vivante : à mettre à jour à chaque migration SQL, policy ou fonction ajoutée/modifiée. Les scripts SQL exécutés manuellement dans le Dashboard ne sont pas versionnés ailleurs — ce fichier est la seule trace écrite.

Projet Supabase : nmqctnqfzqhdgaxneet (URL/clé dans .env.local, NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY).

Tables (schéma public)

État vérifié le 2026-06-23 via `information_schema.tables` : intentions, notifications, participations, prieres, reponses_intention, utilisateurs, pensees_du_jour, videos_motivation.

Les colonnes ci-dessous sont déduites de [src/types/index.ts](#) (source de vérité côté code) et confirmées par les SQL exécutés dans ce projet.

utilisateurs

Profil applicatif, id = même UUID que `auth.users.id`.

colonne	type	notes
id	uuid PK	= <code>auth.uid()</code>
nom, prenom, email	text	
telephone	text, nullable	
role	text	'fidele' \ 'pasteur' \ 'admin'
est_actif	boolean	
date_creation	timestampz	

Politiques (confirmées) : - `utilisateurs_select_own` / `utilisateurs_update_own` : `auth.uid() = id` - `utilisateurs_select_by_admin` / `utilisateurs_update_by_admin` : via la fonction `public.est_admin_ou_pasteur()` (voir plus bas) — **corrigées le 2026-06-22** car la version initiale faisait un SELECT récursif sur `utilisateurs` à l'intérieur de sa propre policy (infinite recursion detected in policy, erreurs 500 en cascade, boucle de redirection login ↔ dashboard).

Création de la ligne profil : double mécanisme constaté le 2026-06-24 — 1. côté client, `register/page.tsx` insère explicitement une ligne juste après `signUp()` (nom/prénom du formulaire, rôle fidele) ; 2. **il existe aussi un trigger côté base** sur `auth.users` (non versionné dans ce repo, configuré directement dans Supabase) qui crée automatiquement une ligne par défaut (nom: "Nouveau", prenom: "Membre", role: "fidele") dès la création d'un compte `auth`, **avant même** l'insert applicatif — d'où un conflit de clé (duplicate key) si on essaie de créer soi-même la ligne après coup (ex. via `auth.admin.createUser`, voir scripts de test). Pour changer le rôle d'un compte créé hors du flow /register (ex. script admin), il faut donc faire un UPDATE sur `utilisateurs`, pas un INSERT. Le détail exact de ce trigger (nom, définition SQL) n'a pas encore été inspecté — à documenter ici si on l'examine un jour via `pg_trigger`.

pensees_du_jour

Créée le 2026-06-23 (table absente jusque-là, voir Historique).

colonne	type	notes
id	uuid PK	<code>gen_random_uuid()</code>
image_url	text	URL publique Storage (bucket pensees)
reference_biblique	text, nullable	accroche / référence biblique
createur_id	uuid	FK → <code>utilisateurs(id)</code>
date_creation	timestampz	défaut <code>now()</code>

Politiques : lecture pour tout authenticated, écriture (insert/update/delete) réservée à

public.est_admin_ou_pasteur()).

videos_motivation

Créée le 2026-06-23 (table absente jusque-là, voir Historique). Couvre en réalité cultes, veillées, camps, retraites (label UI : "Publier une vidéo").

colonne	type	notes
id	uuid PK	
titre	text	
description	text, nullable	
url_video	text	lien YouTube ou URL publique Storage (bucket videos) — distingué côté code par <code>estUrlYoutube()</code> (<code>src/lib/utls.ts</code>), pas de colonne dédiée
createur_id	uuid	FK → utilisateurs(id)
date_creation	timestampz	

Policies : lecture pour tout authenticated, écriture réservée à public.est_admin_ou_pasteur().

prieres

colonne	type	notes
id	uuid PK	
titre, description	text	
type	text	'live' \ 'rediffusion'
url_video	text, nullable	
date_debut, date_fin	timestampz, nullable	
statut	text	'planifie' \ 'en_cours' \ 'termine'
createur_id	uuid	
date_creation	timestampz	

Policies (vérifiées le 2026-06-24 via pg_policies) : - prieres_select_all (SELECT, true) — tout le monde peut lire. - prieres_insert_pasteur (INSERT) / prieres_update_pasteur (UPDATE) — admin/pasteur. - **prieres_delete_pasteur (DELETE) — manquait totalement.** La suppression d'une session de prière échouait silencieusement (RLS bloque par défaut une commande sans policy, sans renvoyer d'erreur exploitée côté code). Ajoutée le 2026-06-24, voir [supabase_fix_policies_manquantes.sql](#).

intentions

colonne	type	notes
id	uuid PK	
user_id	uuid	
sujet, description	text	
statut	text	'en_attente' \ 'en_cours' \ 'traitee'
est_privée	boolean	
date_creation	timestampz	

Policies (vérifiées le 2026-06-24) : - intentions_insert_own (INSERT) / intentions_update_own (UPDATE, auth.uid() = user_id) — le fidèle gère ses propres intentions. - intentions_select_own (SELECT) — soi-même, ou tout pasteur/admin. - **intentions_update_pasteur (UPDATE) — manquait.** CarteIntention.tsx fait passer le statut à traitee depuis le compte du pasteur/admin qui répond (pas le propriétaire de l'intention) : bloqué silencieusement par intentions_update_own. Ajoutée le 2026-06-24 (policy additive, n'enlève rien au fidèle).

reponses_intention

colonne	type	notes
id	uuid PK	
intention_id	uuid	

colonne	type	notes
responsable_id	uuid	
commentaire	text	
date_reponse	timestampz	

Politiques (vérifiées le 2026-06-24) : reponses_insert_pasteur (INSERT), reponses_select_all (SELECT, true). Pas de policy UPDATE/DELETE — cohérent, aucune fonctionnalité d'édition/suppression de réponse dans l'UI actuelle.

participations

colonne	type	notes
id	uuid PK	
user_id, priere_id	uuid	
date_participation	timestampz	
duree_visionnage_sec, position_reprise_sec	int	

Politiques (vérifiées le 2026-06-24) : participations_own, ALL, auth.uid() = user_id — chacun gère uniquement sa propre participation (utilisé par VideoPlayer.tsx en upsert). **participations_select_admin (SELECT) — manquait.** La vue d'ensemble admin agrège les participations de tout le monde (total, % de fidèles engagés, participants au live en cours) en utilisant le client serveur authentifié de l'admin (pas le service role) — sans cette policy additive, RLS limitait silencieusement chaque requête aux seules participations de l'admin lui-même, faussant toutes les statistiques. Ajoutée le 2026-06-24, voir supabase fix participations admin.sql.

live_commentaires

Créée le 2026-06-26, voir supabase_setup_live_commentaires_likes.sql.

colonne	type	notes
id	uuid PK	
priere_id	uuid	FK → prieres(id), on delete cascade
user_id	uuid	FK → utilisateurs(id)
nom_auteur, prenom_auteur	text	dénormalisés : capturés à la publication. Un fidèle ne peut lire que sa propre ligne utilisateurs (RLS), donc impossible de faire un join pour afficher le nom des autres auteurs sans relâcher cette policy.
contenu	text	
date_creation	timestampz	

Politiques : lecture pour tout authenticated, insertion réservée à auth.uid() = user_id, suppression par l'auteur ou est_admin_ou_pasteur() (modération).

live_likes

Créée le 2026-06-26, même script SQL.

colonne	type	notes
id	uuid PK	
priere_id	uuid	FK → prieres(id), on delete cascade
user_id	uuid	FK → utilisateurs(id)
date_creation	timestampz	

Contrainte unique (priere_id, user_id) — un like par fidèle et par session (toggle insert/delete côté client). **Politiques :** lecture pour tout authenticated, insertion/suppression réservées à auth.uid() = user_id.

Realtime non actif par défaut sur une nouvelle table — Supabase n'ajoute pas automatiquement une table à la publication `supabase_realtime` ; sans ça, les écouteurs `postgres_changes` ne reçoivent jamais rien (seule la lecture initiale au chargement fonctionne). Corrigé le 2026-06-27, voir [supabase fix realtime live.sql](#).

Vérification 2026-06-27 : confirmé fonctionnel via un client Node authentifié isolé (l'événement INSERT est bien reçu après le fix). En revanche, les tests automatisés via navigateur headless (Edge piloté par Puppeteer) n'ont jamais reçu l'événement malgré un statut SUBSCRIBED — même en retirant le filtre. La cause la plus probable est une limitation de cet environnement de test headless (WebSocket Realtime), pas un bug du code (qui est structurellement identique à `NotificationBell.tsx`, déjà en production). **À reconfirmer manuellement par l'utilisateur dans deux véritables onglets de navigateur** avant de considérer le sujet clos.

notifications

colonne	type	notes
id	uuid PK	
user_id	uuid	
titre, message	text	
type	text	'info' \ 'live' \ 'intention' \ 'appel'
est_lue	boolean	
date_envoi, date_lecture	timestampz	

Lue en Realtime côté client (`NotificationBell.tsx` — `postgres_changes` sur INSERT). **Politiques (vérifiées le 2026-06-24)** : `notifications_own`, ALL, `auth.uid() = user_id` — chacun ne voit/gère que ses propres notifications. Aucune policy INSERT pour un tiers : si on ajoute un jour une fonctionnalité où l'admin crée des notifications pour d'autres utilisateurs, il faudra une policy `est_admin_ou_pasteur()` dédiée.

Fonctions SQL

`public.est_admin_ou_pasteur()`

```
security definer, stable
-- true si auth.uid() correspond à un utilisateur role admin/pasteur
```

Créée le 2026-06-22 pour remplacer les sous-requêtes récursives dans les politiques de utilisateurs. **Toute policy qui doit vérifier "admin ou pasteur" doit utiliser cette fonction plutôt qu'un SELECT direct sur utilisateurs**, sous peine de retomber dans le même bug de récursion RLS.

Client `service_role` (serveur uniquement)

`src/lib/supabase/admin.ts` crée un client Supabase avec la clé `SUPABASE_SERVICE_ROLE_KEY` (variable d'env **serveur uniquement**, sans préfixe `NEXT_PUBLIC_` — ne jamais l'exposer au navigateur). Ce client contourne RLS entièrement. Utilisé uniquement par :

- `src/app/api/admin/utilisateurs/[id]/route.ts` – DELETE : supprime définitivement un compte (ligne `utilisateurs + auth.users` via `auth.admin.deleteUser`). Vérifie côté serveur que l'appelant est authentifié et admin/pasteur (`getAuthUser/getProfil`) avant d'agir — ne fait confiance à aucune vérification côté client. Un admin ne peut pas se supprimer lui-même (vérification explicite). Ajoutée le 2026-06-24, déclenchée depuis `admin/utilisateurs/page.tsx` (bouton "Supprimer définitivement", à distinguer du simple désactiver/réactiver qui lui passe par le client normal + RLS).

Storage

Bucket `pensees`

Public, créé le 2026-06-23. Politiques : lecture publique, upload réservé à `est_admin_ou_pasteur()`. Chemin des fichiers : `{userId}/{timestamp}-{nom}`.

Bucket `videos`

Public, créé le 2026-06-23 ([supabase_setup_videos_storage.sql](#)). Mêmes politiques que `pensees`. Utilisé par le

formulaire de publication de vidéo ([FormulaireVideo.tsx](#)) quand l'admin/pasteur choisit "Importer un fichier" plutôt qu'un lien YouTube. Limite de taille de fichier par défaut Supabase (~50 Mo selon le plan) : privilégier YouTube pour les vidéos longues (cultes entiers).

Historique des incidents / correctifs

date	problème	cause	correctif
2026-06-21	Connexion impossible ("Failed to fetch")	Latence/instabilité réseau transitoire vers Supabase	aucun changement de code nécessaire
2026-06-21	"Email not confirmed" au login	Confirmation email Supabase activée, compte jamais confirmé	confirmation manuelle dans le Dashboard / désactivation de l'option en dev
2026-06-21	Dashboard tout noir, boucle de redirection	proxy.ts traitait une erreur réseau de getUser() comme "non connecté"	proxy.ts laisse passer la requête si error est définie au lieu de rediriger
2026-06-22	ERR_TOO_MANY_REDIRECTS login ↔ dashboard	Policies RLS récursives sur utilisateurs → 500 sur select role → fallback /dashboard côté proxy/login, mais getProfil() échouait aussi → redirection vers /login → boucle	création de est_admin_ou_pasteur() (SECURITY DEFINER), mais policies réécrites avec cette fonction
2026-06-23	alter table pensees_du_jour échoue : relation does not exist	Les tables pensees_du_jour et videos_motivation n'avaient jamais été créées en base (le code supposait leur existence)	script de création complet (tables + RLS + bucket Storage), voir supabase_setup_pensees_videos.sql
2026-06-23	Formulaire vidéo limité à un lien YouTube	Pas d'option d'import direct depuis téléphone/PC	ajout d'un choix de source (YouTube ou fichier) dans FormulaireVideo.tsx, nouveau bucket Storage videos, détection du type d'URL côté affichage via estUrlYoutube()

date	problème	cause	correctif
2026-06-24	Suppression d'une session de prière sans effet (pas d'erreur visible)	prieres n'avait aucune policy DELETE (RLS bloqué silencieusement) + le code ignorait l'error retourné par .delete()	ajout de prieres_delete_pasteur , et ajout de la gestion d'erreur visible sur tous les handlers CRUD qui l'ignoraient (admin/prieres, admin/videos, admin/utilisateurs, CarteIntention.tsx) ; ajout aussi de intentions_update_pasteur (bug identique : un admin/pasteur ne pouvait pas faire passer une intention à traitee)
2026-06-24	Stats admin (vue d'ensemble) potentiellement fausses pour les participations	participations n'avait qu'une policy ALL scopée à l'admin lui-même	ajout de participations_select_admin, voir supabase_fix_participations_admin.sql
2026-06-24	Pas de moyen de supprimer définitivement un compte (seulement désactiver)	Suppression d'un compte auth nécessite la clé service_role, jamais exposable côté client	nouvelle route serveur api/admin/utilisateurs/[id] (DELETE) + src/lib/supabase/admin.ts, bouton dédié dans admin/utilisateurs/page.tsx
2026-06-26	Demande : que "Démarrer un live" diffuse réellement webcam/micro vers les fidèles, avec compteur de connectés/commentaires/likes	Diffuser une vraie webcam vers plusieurs viewers nécessite un serveur de streaming (SFU/RTMP→HLS) que Next.js/Supabase ne fournissent pas – hors de portée sans service tiers	choix utilisateur : garder le flux YouTube Live existant (admin diffuse via téléphone/OBS vers YouTube, l'app stocke le lien) ; ajout de DemarrerLiveModal, d'un bouton "Terminer le live" (statut=termine + type=rediffusion), et de LiveInteractions (présence Realtime pour le nombre de connectés + tables live_likes/live_commentaires)

date	problème	cause	correctif
2026-06-29	Demande : doc Swagger/OpenAPI du backend, accessible et – partageable, à tenir à jour		création de public/openapi.yaml (toutes les tables Supabase en endpoints REST + la route DELETE /api/admin/utilisateurs/{id}) et de public/api-docs.html (Swagger UI via CDN, sans dépendance npm), accessible sur /api-docs.html ; lien ajouté en bas de /admin
2026-06-29	Audit responsive : actions de la navbar (Paramètres/Notifications/Déconnexion) invisibles sur mobile ; + actions des tableaux admin (utilisateurs/prières/vidéos) totalement inaccessibles sur mobile	overflow-hidden du wrapper (dashboard)/layout.tsx (mis en place pour clipper les halos décoratifs) clippait aussi le débordement horizontal de la navbar ; les tableaux n'avaient aucun mécanisme de scroll horizontal	Navbar.tsx/AdminNav.tsx : liens nav dans un conteneur flex-1 min-w-0 overflow-x-auto, actions en shrink-0 (toujours visibles) ; tableaux admin enveloppés dans un div.overflow-x-auto avec min-w-* sur <table>
2026-06-29	Mot de passe oublié : implémentation + lien depuis /login	@supabase/ssr force le flow pkce côté client, incompatible avec le lien #access_token= du template d'email par défaut Supabase – session de récupération jamais détectée	nouvelle route serveur auth/confirm (verifyOtp + cookies), pages /mot-de-passe-oublie et /reinitialiser-mot-de-passe. Nécessite de modifier le template "Reset Password" dans le Dashboard, voir détail juste en dessous

Détail : action requise pour le mot de passe oublié

Le template d'email "Reset Password" (Supabase Dashboard → Authentication → Email Templates) utilise par défaut {{ .ConfirmationURL }}, qui produit un lien à fragment (#access_token=...&type=recovery) — incompatible avec le flow PKCE forcé par @supabase/ssr. Il faut remplacer ce lien par :

```
{{ .SiteURL }}/auth/confirm?token_hash={{ .TokenHash }}&type=recovery&next=/reinitialiser-mot-de-passe
```

Vérifié de bout en bout via auth.admin.generateLink() (expose directement hashed_token, permet de tester sans attendre un vrai email) : token_hash → /auth/confirm → session établie via cookies → nouveau mot de passe → reconnexion réussie avec le nouveau mot de passe.